Software Architecture and Design Specification

Project: Electronics Store Inventory Management System (EIMS)

Version: 1.0

Authors: Team 9 – Karan Maheshwari, Krishitha Kankanala, Jayashree D, Hemavathi E

Date: 16-09-2026

Revision History

Version	Date	Author	Change Summary
1.0	16-09-2026	Team 9	Initial SAD prepared from the EIMS project baseline and STP.

Approvals

Role	Name	Signature / Date
QA Lead	Hemavathi E	
Dev Lead	Krishitha Kankanala	
Product Owner / Project Team Lead	Karan Maheshwari	
Project Guide / Evaluator	Prof. M. Sheela Devi	

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design for the Electronics Store Inventory Management System (EIMS) v1.0. It translates the functional, non-functional and security requirements described in the project baseline into an implementable architecture, component structure, interaction flows, security controls and design decisions.

1.2 Scope

EIMS is a standalone web-based application for managing electronics-store component inventory. The design covers inventory viewing, search, filtering, sorting, authorized updates, adding components, deletion/discontinuation, availability reporting, dashboard functions, authentication, role-based access control, audit logging and the required security, performance, scalability, usability and responsiveness characteristics. Point-of-sale payment-gateway integration and supplier-side procurement are outside the current scope.

1.3 Audience

The intended audience is the development team, QA/test team, project guide/evaluator, store-management stakeholders and anyone responsible for security review, deployment or maintenance.

1.4 Definitions

Term	Meaning
EIMS	Electronics Store Inventory Management System
SRS	Software Requirements Specification
STP	Software Test Plan
UI	User Interface
API	Application Programming Interface
DB	Database
SKU	Stock Keeping Unit
CRUD	Create, Read, Update, Delete
RBAC	Role-Based Access Control
JWT	JSON Web Token
REST	Representational State Transfer
NFR	Non-Functional Requirement
RTM	Requirements Traceability Matrix

2. Document Overview

2.1 How to use this document

The document is organized from high-level architecture to detailed design. Sections 3 and 4 describe the architecture, components, technology assumptions, security structure, UML interactions, interfaces, error handling and user experience. The appendices provide traceability, architecture decisions, glossary and references. The component and sequence diagrams are intended to be used together with the requirement IDs and test references in the project STP.

2.2 Related Documents

- Electronics Store Inventory Management System Software Test Plan (STP), Version 1.0, dated 16-09-2026.
- Electronics Store Inventory Management System SRS v1.0, dated 06-09-2026, as referenced by the STP.
- Requirements Traceability Matrix represented in the STP.

3. Architecture

3.1 Goals & Constraints

Area	Architecture goal / constraint
Security	Authentication is required before protected inventory functions; RBAC separates Admin/Store Manager and Staff; HTTPS/TLS 1.2+ is required; input must be validated and sanitized.
Performance	Dashboard and inventory pages should load within 3 seconds for 90% of normal-load requests; search results are expected within 2 seconds.
Scalability	The system should support at least 10,000 components and 20 concurrent users without performance degradation.

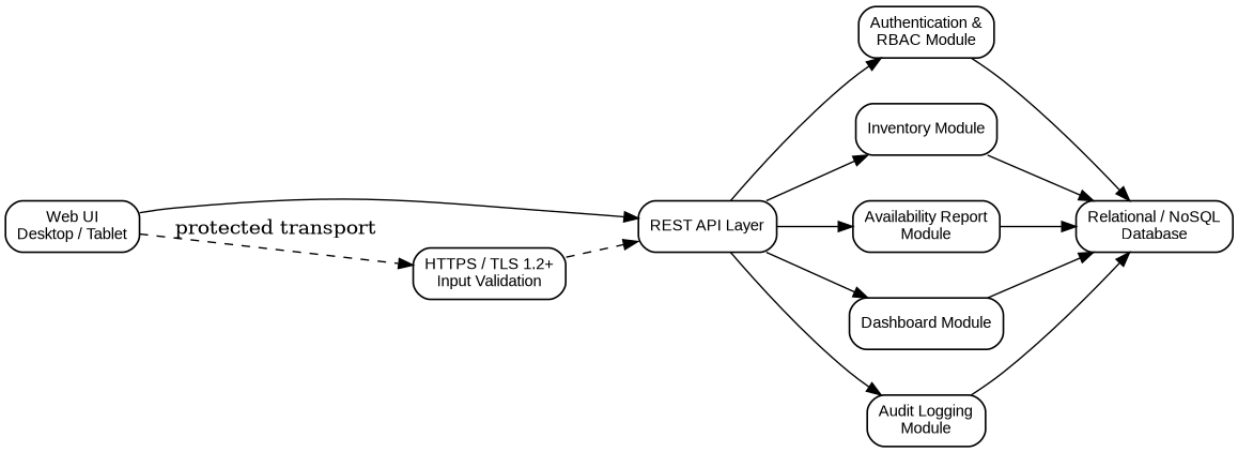
Area	Architecture goal / constraint
Usability	A new staff member should be able to add or search for a component within five minutes without formal training.
Maintainability	The codebase should separate UI, API and data-access responsibilities and remain documented for maintenance.
Portability	The web UI should remain usable on desktop and tablet screen sizes.
Scope constraint	POS payment-gateway integration and supplier-side procurement are excluded from the current release.

3.2 Stakeholders & Concerns

Stakeholder	Primary concern
Store Manager / Admin	Inventory accuracy, authorized updates/deletion, reports, dashboard and security
Store Staff	Fast inventory search, viewing, filtering and role-appropriate dashboard access
QA / Security	Requirement coverage, RBAC, input validation, HTTPS, session expiry and auditability
Project Team / Guide	Traceability, maintainability, architecture clarity and acceptance readiness

3.3 Component (UML) Diagram

The component structure below is derived from the STP architecture description: web front end, REST API server, persistence layer, authentication/RBAC, inventory, reporting, dashboard and audit logging. The exact database technology is intentionally left open because the project baseline permits a relational or NoSQL database selected by the development team.



3.4 Component Descriptions

Component	Responsibility	Key requirement coverage
Web UI	Presents inventory, add/delete forms, reports, dashboard and role-appropriate controls; supports desktop/tablet views.	F-001 to F-022; NF-003, NF-006

Component	Responsibility	Key requirement coverage
REST API Layer	Exposes application operations to the UI and coordinates authentication, inventory, reports and dashboard requests.	F-001 to F-022; SR-001, SR-003, SR-004, SR-006
Authentication & RBAC	Authenticates users, manages protected sessions and enforces Admin/Manager versus Staff permissions.	SR-001, SR-003, SR-005
Inventory Module	Handles component listing, search, filtering, sorting, creation, validation, updates and deletion.	F-001 to F-014
Availability Report Module	Classifies inventory as in-stock, low-stock or out-of-stock; supports filters and export where implemented.	F-015 to F-018
Dashboard Module	Calculates and presents component totals, stock value, low-stock count, recent items, charts and low-stock alerts.	F-019 to F-022
Audit Logging Module	Records user, timestamp and component for inventory deletion events.	F-014
Database	Persists components and related category, supplier, user and audit information required by the implementation.	F-001 to F-022; SR-002

3.5 Chosen Architecture Pattern and Rationale

A layered architecture is selected because the STP explicitly describes EIMS as a standalone web application with a front-end interface, back-end REST API server and relational/NoSQL database. Separating presentation, API/application logic and data access supports the maintainability requirement EIMS-NF-005 and makes security and role checks consistent across protected operations. A microservices decomposition is not required by the project baseline and would add deployment and operational complexity for the stated scope.

3.6 Technology Stack & Data Stores

The project baseline specifies observable architecture rather than a mandatory framework choice. The design therefore uses the following implementation-neutral stack description: web browser client; REST API over HTTPS; relational or NoSQL persistence selected by the development team; and a data-access layer for component, category, supplier, user and audit information. Modern Chrome, Firefox or Edge browsers are identified for testing. If the team selects a concrete framework such as Django/DRF or MERN, it should remain an implementation detail behind the architecture described here.

3.7 Risks & Mitigations

Risk	Mitigation
Stable build delayed	Prepare test data, architecture and test cases in parallel and request early smoke builds.
Database/API unavailable	Maintain a repeatable test deployment and preserve prepared test data.
Incorrect role configuration	Use separate Admin/Manager and Staff accounts and verify permissions before RBAC tests.
Large dataset affects results	Prepare the 10,000-component dataset in advance and record environment limitations separately.
Security payloads affect data	Use isolated test accounts and non-production data for injection/XSS and session-expiry checks.

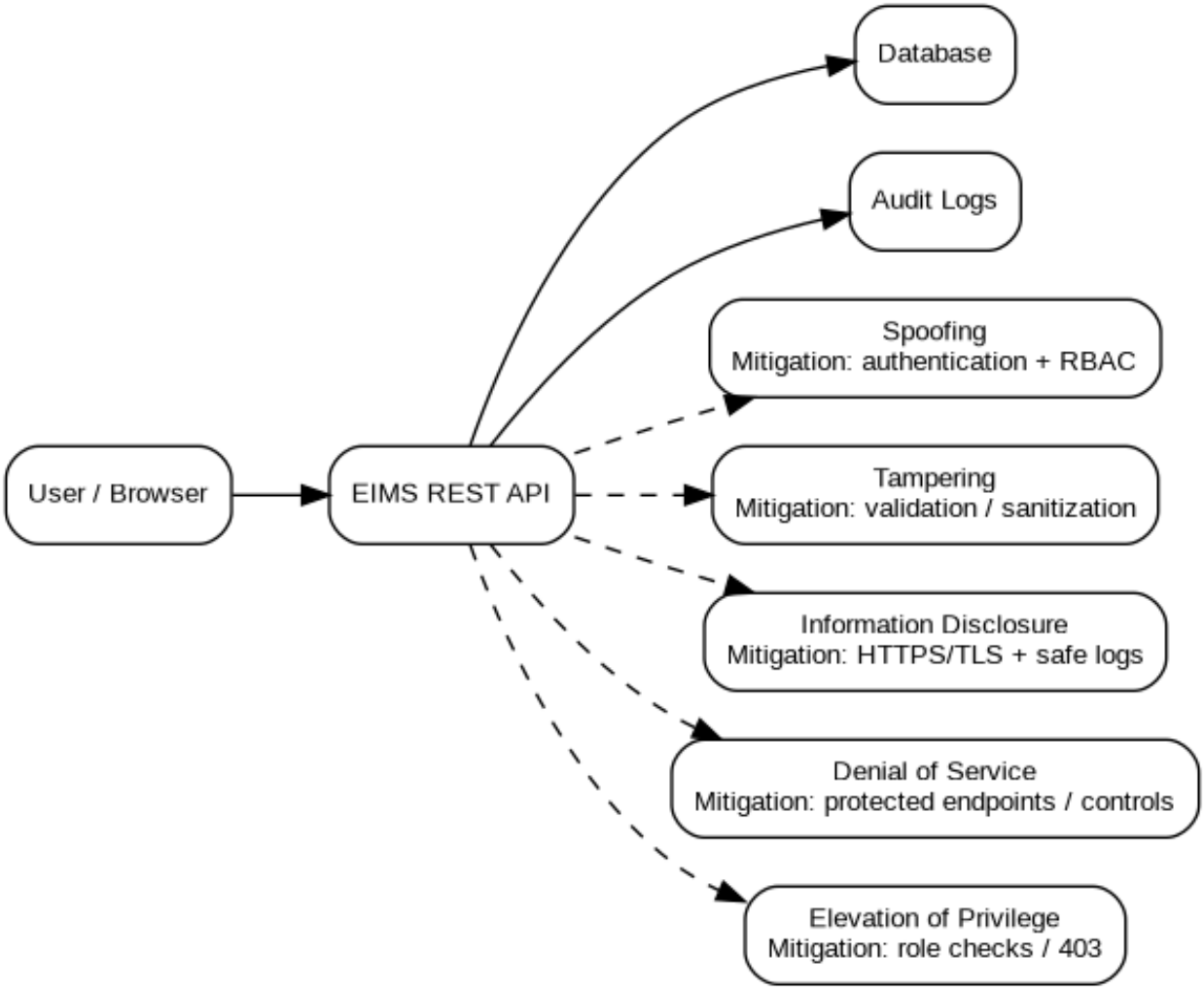
Risk	Mitigation
Shared inventory data becomes inconsistent during testing	Use controlled records and reset/restore data between tests when required.

3.8 Traceability to Requirements

Requirement group	Architecture mapping
EIMS-F-001 to F-005	Inventory Module + Web UI + REST API + Database
EIMS-F-006 to F-010	Inventory Module + validation + Database + Web UI
EIMS-F-011 to F-014	Inventory Module + Authentication/RBAC + Audit Logging + Database
EIMS-F-015 to F-018	Availability Report Module + Database + Web UI
EIMS-F-019 to F-022	Dashboard Module + Database + REST API + Web UI
EIMS-NF-001 to NF-006	Web UI, API, database and modular component boundaries
EIMS-SR-001 to SR-006	Authentication/RBAC, HTTPS/TLS, session management, input validation and data-access controls

3.9 Security Architecture

Security architecture follows the six security requirements in the project baseline. Authentication protects inventory functions and protected requests are rejected when unauthenticated. RBAC distinguishes Admin/Manager and Staff; Staff access to restricted delete/admin endpoints must return HTTP 403. Passwords must be stored as salted one-way hashes. Client-server communication uses HTTPS with TLS 1.2 or later. Inactive sessions terminate after 15 minutes. Input is validated and sanitized to reduce SQL/NoSQL injection and XSS risk.



Threat category	EIMS concern	Design control
Spoofing	Unauthenticated access to protected inventory functions	Authentication and session validation
Tampering	Malicious or malformed inventory/user input	Validation and sanitization; protected API operations
Information disclosure	Credentials, account or inventory data exposed in transit/logs	HTTPS/TLS 1.2+; no sensitive information in logs
Denial of Service	Excessive or disruptive requests affecting availability	Protected endpoints, request controls and monitoring
Elevation of Privilege	Staff attempting Admin/Manager actions	RBAC checks and HTTP 403 for restricted endpoints

4. Design

4.1 Design Overview

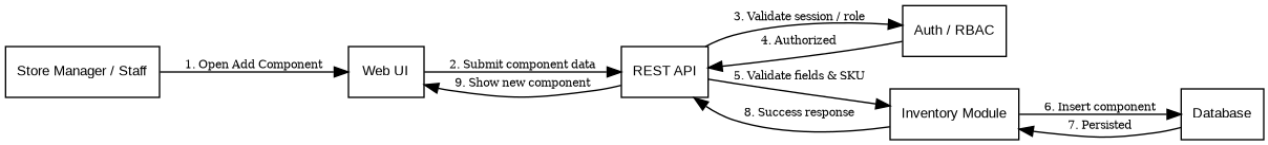
EIMS is designed as a layered web application. The Web UI is responsible for presentation and user interaction. The REST API provides a controlled application boundary. Authentication and RBAC are applied before protected operations. Inventory, report, dashboard and audit modules contain their respective business responsibilities. The

data-access layer persists system information and supports consistency between inventory, reports and dashboard views.

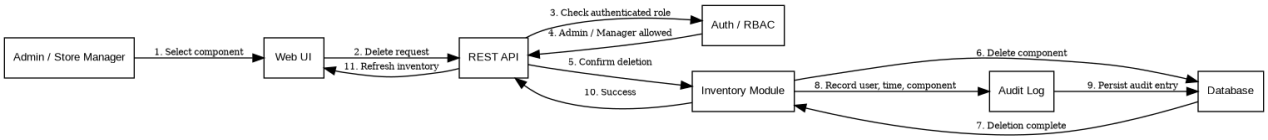
4.2 UML Sequence Diagrams

The following flows are selected directly from the functional requirements in the STP. The first covers adding a component (F-006 to F-010). The second covers authorized deletion and mandatory audit logging (F-011 to F-014).

4.2.1 Add Component



4.2.2 Delete Component with Audit Logging



4.3 API Design

The STP requires a REST API architecture but does not prescribe concrete endpoint names. The following interfaces are therefore design-level endpoint proposals mapped to the externally observable requirements. They can be implemented with the framework selected by the development team.

Interface	Method	Request / Inputs	Response / Behaviour	Security
Authenticate user	POST /api/auth/login	username, password	Authenticated session/token; invalid credentials rejected	SR-001, SR-005
List/search inventory	GET /api/components	search, category, supplier, stock status, sort parameters	Active inventory records matching requested criteria	Authenticated users
Add component	POST /api/components	name, category, quantity, unit price, supplier; optional image	Created component with unique SKU; validation errors returned clearly	Authenticated; role per implementation
Update component	PATCH /api/components/{id}	quantity, price, supplier, reorder level	Updated record persisted immediately	Authorized user; SR-003
Delete component	DELETE /api/components/{id}	component identifier	Deletion result; audit record created	Admin/Manager only; 403 for Staff
Availability report	GET /api/reports/availability	category, supplier, date range, export option	In/low/out-of-stock classification and export where implemented	Authenticated
Dashboard	GET /api/dashboard	none or dashboard filters	Totals, stock value, low-stock count, recent items, chart data, alerts	Role-based view

4.4 Error Handling, Logging & Monitoring

- Use clear validation messages when mandatory or invalid fields are submitted (F-008).
- Reject unauthorized delete/admin requests with HTTP 403 where specified by SR-003; reject unauthenticated protected requests with HTTP 401 under SR-001.
- Handle duplicate SKU attempts as a controlled validation/business-rule error (F-009).
- Do not store passwords or sensitive authentication material in application logs.
- Record deletion audit events with user, timestamp and component information (F-014).

• Monitor request response times, inventory/report/dashboard failures, authentication failures, low-stock alert behaviour and overall availability.

• Use the STP-defined thresholds for verification: 3-second page-load criterion, 2-second search criterion, 5-second low-stock alert criterion and 15-minute inactive-session termination.

4.5 UX Design

The UI should present inventory information in a straightforward list with name, SKU, category, quantity, unit price, supplier and reorder level. Search, filtering and sorting should be available from the inventory view. Add and delete actions should provide clear validation and confirmation feedback. Dashboard views should be role-based: the Admin/Manager view can expose the full dashboard, while Staff receives a read-only summary. The interface should remain usable on desktop and tablet screen sizes and should support the five-minute new-staff usability criterion.

4.6 Open Issues & Next Steps

• Select and document the final database technology and concrete application framework during implementation; the STP intentionally does not mandate Django/DRF, MERN or another stack.

• Confirm which optional interfaces are included in the release: barcode scanner, printer, notification service and PDF/Excel/CSV export.

• Define the exact token/session mechanism and retention configuration while preserving the 15-minute inactivity requirement.

• Create the final deployment topology and operational availability measurement needed to verify the 99.5% monthly availability requirement.

• Run the security scan and performance/load tests in the production-like test environment described by the STP.

Appendix A – Architecture Decision Records (ADRs)

ADR	Decision	Rationale	Status
ADR-001	Use layered web architecture	Matches the documented EIMS structure of Web UI, REST API and database; supports modularity and maintainability.	Accepted for design
ADR-002	Keep database technology implementation-neutral	The STP allows a relational/NoSQL database and says testing should not impose a technology choice.	Accepted for design
ADR-003	Centralize authentication and RBAC at the API boundary	Protected inventory functions require authentication and role restrictions must be consistent across UI and API.	Accepted for design
ADR-004	Create audit records for authorized deletion	F-014 explicitly requires user, timestamp and component to be recorded for every deletion.	Accepted for design
ADR-005	Use HTTPS/TLS 1.2+ for client-server communication	SR-004 requires TLS 1.2+ and plain HTTP must be redirected or rejected.	Required by baseline

Appendix B – Requirement-to-Design Traceability

Req ID	Requirement short	Design component	Test reference
EIMS-F-001	View inventory list	InventoryModule / WebUI	TC-Inv-01
EIMS-F-002	Search by name/SKU/category	InventoryModule / WebUI	TC-Inv-02
EIMS-F-003	Filter by category/supplier/status	InventoryModule / WebUI	TC-Inv-03
EIMS-F-004	Sort by quantity/price/date	InventoryModule / WebUI	TC-Inv-04
EIMS-F-005	Update component details	InventoryModule / DB	TC-Inv-05
EIMS-F-006	Add new component	InventoryModule / WebUI	TC-Add-01
EIMS-F-007	Unique SKU generation	InventoryModule / DB	TC-Add-02
EIMS-F-008	Mandatory-field validation	InventoryModule / API	TC-Add-03
EIMS-F-009	Duplicate SKU prevention	InventoryModule / DB	TC-Add-04
EIMS-F-010	Optional image upload	WebUI / API	TC-Add-05
EIMS-F-011	Delete component	InventoryModule	TC-Del-01
EIMS-F-012	Deletion confirmation	WebUI	TC-Del-02
EIMS-F-013	Delete role restriction	AuthModule	TC-Del-03
EIMS-F-014	Deletion audit log	AuditModule	TC-Del-04
EIMS-F-015	Availability report	ReportModule	TC-Rep-01
EIMS-F-016	Filter availability report	ReportModule	TC-Rep-02
EIMS-F-017	Export report	ReportModule	TC-Rep-03
EIMS-F-018	Highlight low-stock	ReportModule	TC-Rep-04
EIMS-F-019	Dashboard summary	DashboardModule	TC-Dash-01
EIMS-F-020	Dashboard charts	DashboardModule	TC-Dash-02
EIMS-F-021	Role-based dashboard	Dashboard/AuthModule	TC-Dash-03
EIMS-F-022	Real-time low-stock alert	DashboardModule	TC-Dash-04
EIMS-NF-001	Page-load performance	WebUI / API	TC-Perf-01
EIMS-NF-002	Monthly availability	Operations	Ops reports
EIMS-NF-003	New staff usability	WebUI	TC-UX-01
EIMS-NF-004	10,000 components / 20 users	WebUI / API / DB	TC-Scale-01
EIMS-NF-005	Modular codebase/documentation	Codebase	Code review checklist
EIMS-NF-006	Desktop/tablet responsiveness	WebUI	TC-UX-02
EIMS-SR-001	Authentication required	AuthModule	TC-Sec-01
EIMS-SR-002	Salted one-way password hash	AuthModule / DB	TC-Sec-02
EIMS-SR-003	RBAC	AuthModule	TC-Sec-03
EIMS-SR-004	TLS 1.2+ HTTPS	Web/API	TC-Sec-04

Req ID	Requirement short	Design component	Test reference
EIMS-SR-005	15-minute inactive session termination	AuthModule	TC-Sec-05
EIMS-SR-006	Input sanitization / injection / XSS	Web/API	TC-Sec-06

Appendix C – Glossary

Term	Definition
EIMS	Electronics Store Inventory Management System
RBAC	Role-Based Access Control
REST	Representational State Transfer
SKU	Stock Keeping Unit
JWT	JSON Web Token
TLS	Transport Layer Security
RTM	Requirements Traceability Matrix
NFR	Non-Functional Requirement
UAT	User Acceptance Testing
OWASP ZAP	Security testing/scanning tool referenced by the STP

Appendix D – References & Tools

- Electronics Store Inventory Management System STP v1.0, 16-09-2026.
- Electronics Store Inventory Management System SRS v1.0, 06-09-2026, referenced by the STP.
- IEEE 42010, OWASP and NIST SP 800-160 are listed as reference sources in the supplied SAD template.
- PlantUML / draw.io may be used for maintaining UML diagrams; Swagger/OpenAPI may be used to document the REST API.